

CHARACTER RECOGNITION NEURAL NETWORK

From-Scratch NumPy Implementation and Evaluation

Author	Joewin Paul Kumar M
Project	35-Class Handwritten Character Recognition
Implementation	Python + NumPy only (no ML frameworks)
Purpose	Final project report for internship screening

Project focus: data preparation, L-layer neural-network implementation, verification, optimization, regularization, and error analysis.

1. Executive Summary

This project implements a 35-class handwritten character recognition system completely from scratch using NumPy. The target classes are uppercase letters A–Z and digits 1–9. The implementation covers the complete machine-learning workflow: dataset loading, preprocessing, stratified splitting, mini-batch training, forward propagation, backpropagation, activation functions, softmax output, cross-entropy loss, multiple optimization methods, L2 regularization, validation, and final test evaluation.

The final dataset contains 10,500 samples, exactly 300 per class. A 70/15/15 stratified split produces 7,350 training, 1,575 validation, and 1,575 test examples, with exactly 210/45/45 samples per class. The final architecture is $784 \rightarrow 128 \rightarrow 64 \rightarrow 35$, using ReLU in the hidden layers and softmax in the output layer.

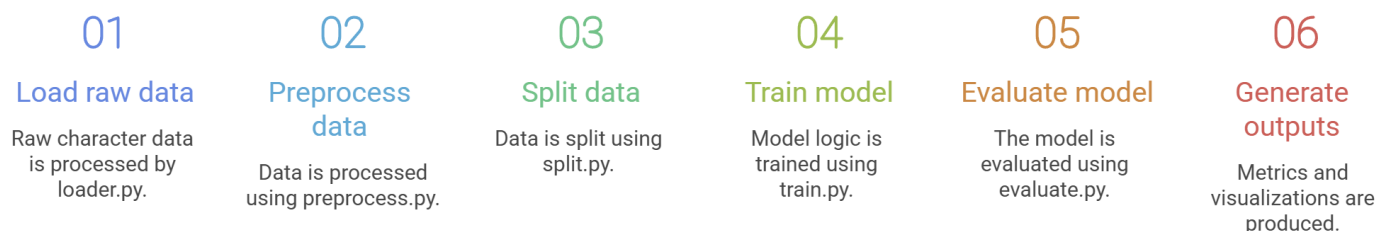
An important implementation decision was to make the network generic for L layers instead of hardcoding a three-layer model. This allows the same initialization, forward, backward, and optimization code to work with different depths simply by changing the layer-dimension list. This improved the extensibility of the project while keeping the actual architecture used for the reported experiment fixed at $784 \rightarrow 128 \rightarrow 64 \rightarrow 35$.

2. Assignment Alignment

Requirement	Implementation	Status
35-class recognition	A–Z and 1–9, 35 total classes	Completed
At least 50 samples/class	300 samples/class, 10,500 total	Exceeded
Preprocessing	28×28 input, normalization, flattening, one-hot labels	Completed
Train/validation/test split	70/15/15 stratified; 210/45/45 per class	Completed

Requirement	Implementation	Status
NumPy-only neural network	No TensorFlow, PyTorch, Keras, or sklearn training	Completed
Forward propagation	Generic L-layer implementation	Completed
Backpropagation	Generic L-layer implementation + gradient checking	Completed
Activation functions	ReLU hidden layers; ReLU derivative; softmax output	Completed
Cross-entropy loss	Implemented from scratch	Completed
Gradient descent	Implemented, then extended to Momentum/RMSprop/Adam	Completed
Validation loop	Validation accuracy measured after each epoch	Completed
Metrics	Accuracy, precision, recall, F1, macro average	Completed
Confusion matrix	Computed and visualized	Completed
Five incorrect predictions	Five cases analyzed with confidence and likely causes	Completed

3. Project Architecture and Data Flow



4. Dataset and Data Preparation

4.1 Data source and composition

The dataset combines established handwriting sources for the required character set. The final CSV contains uppercase letters A–Z and digits 1–9 only; digit 0 is intentionally excluded. The final dataset contains 10,500 samples: 35 classes × 300 samples per class.

Class balance was explicitly verified with NumPy rather than assumed. This matters because balanced data makes accuracy and macro/weighted averages easier to interpret.

4.2 Preprocessing

- Input validation: the preprocessing module accepts image arrays and guarantees a single-channel representation when required.
- Fixed image size: the dataset already contains 28×28 images, so the implemented resize operation returns the input unchanged for the final dataset. The actual conversion from 28×28 into the neural-network input size of 784 is performed by flattening, not resizing.
- Normalization: pixel values are divided by 255 so the model receives values in the range [0, 1].
- Flattening: each 28×28 image becomes a 784-element vector because $28 \times 28 = 784$.
- One-hot encoding: each 1–35 class label is converted into a 35-element vector containing one 1 and thirty-four 0s.

4.3 Why one-hot encoding was used

The 35 classes are categorical, not ordered numerical quantities. Representing class 7 as the number 7 could incorrectly suggest that class 7 is mathematically larger than class 3. One-hot encoding removes this false ordering. For example, for three classes, class 1, class 2, and class 3 become [1,0,0], [0,1,0], and [0,0,1]. This representation also matches the 35-neuron softmax output layer.

4.4 Data orientation

The final implementation uses the convention $X = (\text{features}, \text{samples})$ and $Y = (\text{classes}, \text{samples})$. Therefore, X has shape (784, m) and Y has shape (35, m). This convention was applied consistently across batching, forward propagation, backpropagation, softmax, loss calculation, and evaluation.

4.5 Stratified splitting

Each class is split independently at 70% training, 15% validation, and 15% test before the class-wise subsets are recombined and shuffled. Because every class contains 300 samples, the exact result is 210/45/45 per class. The final totals are 7,350 training samples, 1,575 validation samples, and 1,575 test samples.

4.6 Random seed and reproducibility

A random seed does not remove randomness; it makes a pseudo-random process repeatable. For example, if the sample indexes are shuffled with seed 42, rerunning the same code with seed 42 produces the same shuffle. This was used for dataset splitting, parameter initialization, and mini-batch ordering so that experiments could be reproduced.

4.7 Build-once, reuse-everywhere approach

A dedicated `build_dataset.py` script loads the raw CSV, preprocesses it, creates the stratified split, and saves the six processed arrays as `.npz` files. Training and evaluation then load these processed arrays directly. This avoids repeatedly rebuilding the dataset and makes downstream experiments faster and easier to reproduce.

5. Neural Network Architecture

Final reported architecture:

784 → 128 → 64 → 35

The 784 inputs represent the flattened 28×28 image. The two hidden layers contain 128 and 64 neurons, and the output layer contains 35 neurons, one for each class.

5.1 Why the implementation was generalized to L layers

The assignment required a neural network, but the implementation was intentionally not hardcoded as “three layers.” Instead, the architecture is supplied as a list such as [784, 128, 64, 35]. Initialization, forward propagation, backward propagation, and parameter updates loop over this list. As a result, the same code can represent a 2-layer, 3-layer, or deeper network without rewriting the mathematical logic for each layer.

This design was chosen because the mathematical operation is the same at every dense layer: $Z = W \cdot A_{\text{prev}} + b$. Only the activation of the final layer changes from ReLU to softmax. Generalizing the implementation therefore reduces repeated code, makes debugging easier, and allows future experiments with different architectures by changing one configuration list.

5.2 Weight initialization

Hidden ReLU layers use He initialization, with weights sampled using a scale of $\sqrt{2/n_{\text{in}}}$. The output layer uses a Xavier-style scale of $\sqrt{1/n_{\text{in}}}$. Biases are initialized to zero. A fixed random seed makes the initial parameters reproducible.

6. Forward Propagation

For every layer l , the network first computes:

$$Z[l] = W[l] \cdot A[l-1] + b[l]$$

Hidden layers then apply ReLU. The output layer applies softmax so that the 35 raw output scores become class probabilities that sum to one for each example.

6.1 ReLU

ReLU is $f(x) = \max(0, x)$. It passes positive values through and replaces negative values with zero. Its derivative is 0 for $x \leq 0$ and 1 for $x > 0$. This derivative is used during backpropagation.

6.2 Softmax and numerical stability

Softmax converts the final scores into probabilities. The implementation subtracts the maximum score from each example before applying the exponential. This does

not change the final probabilities, but it prevents very large exponential values and therefore reduces the risk of numerical overflow.

7. Loss Function

Cross-entropy loss measures how far the predicted probability distribution is from the correct one-hot label. A correct class with high predicted probability produces a low loss; assigning a low probability to the correct class produces a higher loss.

$$\text{Loss} = -(1/m) \sum Y \cdot \log(Y_{\text{hat}})$$

8. Backpropagation

The backward pass starts at the output and moves backward one layer at a time. For softmax combined with cross-entropy, the final error simplifies to:

$$dZ[L] = A[L] - Y$$

For hidden ReLU layers, the error is passed through the next layer weights and then multiplied element-wise by the ReLU derivative. Weight and bias gradients are calculated for every layer and returned to the optimizer.

8.1 Gradient verification

Two levels of verification were used. First, gradient shapes were checked against the corresponding parameter shapes. Second, analytical backpropagation gradients were compared with centered finite-difference gradients. The numerical check used a smaller network for practical runtime reasons, while the same generic backward-pass code path was exercised. The reported relative error was approximately $1e-8$, comfortably below the $1e-5$ acceptance threshold used for the project. For the largest $W1$ matrix, 50 elements were sampled while smaller parameter arrays were checked exhaustively.

9. Optimization and Mini-Batch Training

9.1 Mini-batch training

Instead of sending the complete training dataset through the network for every update, the training set is divided into smaller batches. The final implementation

uses a batch size of 64. Each batch follows the same sequence: forward pass → loss → backward pass → optimizer update.

Mini-batching reduces the memory required for a single update and allows the model to update its parameters many times during one epoch. The sample order is reshuffled each epoch using a reproducible seed tied to the epoch number.

9.2 Optimizers implemented

Optimizer	Main idea	Role in project
Gradient Descent	$W = W - \text{learning_rate} \times dW$	Baseline optimizer and first implementation
Momentum	Uses a moving average of past gradients	Smoother updates
RMSprop	Uses a moving average of squared gradients	Adaptive step size per parameter
Adam	Combines momentum and RMSprop with bias correction	Final optimizer used for training

The Adam step counter t is incremented once per batch and is not reset at the beginning of an epoch. This is necessary because Adam's bias-correction factors depend on the total number of update steps.

10. Regularization and Overfitting

The unregularized baseline reached 98.9% training accuracy but only 81.1% validation accuracy after 50 epochs, giving a 17.8 percentage-point gap. This was treated as evidence of overfitting rather than as a satisfactory final model.

L2 regularization was then added as a separate, verified step. The additional loss term is proportional to the sum of squared weight values, while the gradient is adjusted by $(\lambda/m)W$. Biases are not regularized.

Learning Rate	Train Accuracy	Validation Accuracy	Gap
0	98.9%	81.1%	17.8
0.1	96.0%	80.9%	15.1
0.5	85.9%	80.4%	5.5
1.0	77.9%	74.3%	3.6

The $\lambda = 0.5$ configuration was selected for the final reported experiment because it reduced the train-validation gap substantially while keeping validation accuracy close to the unregularized baseline. $\lambda = 1.0$ reduced the gap further but also lowered validation accuracy, indicating over-regularization and underfitting.

11. Training and Validation Results

The training loop repeats the following steps for each mini-batch:

1. Load a mini-batch of 64 samples.
2. Run forward propagation.
3. Compute cross-entropy loss and, for the regularized experiment, the L2 penalty.
4. Run backpropagation.
5. Add the L2 gradient term to the weight gradients when regularization is enabled.
6. Update all weights and biases with Adam.

After all batches in an epoch, training and validation accuracy are calculated. This makes it possible to observe both learning progress and the train-validation gap during development.

12. Final Test Set Evaluation

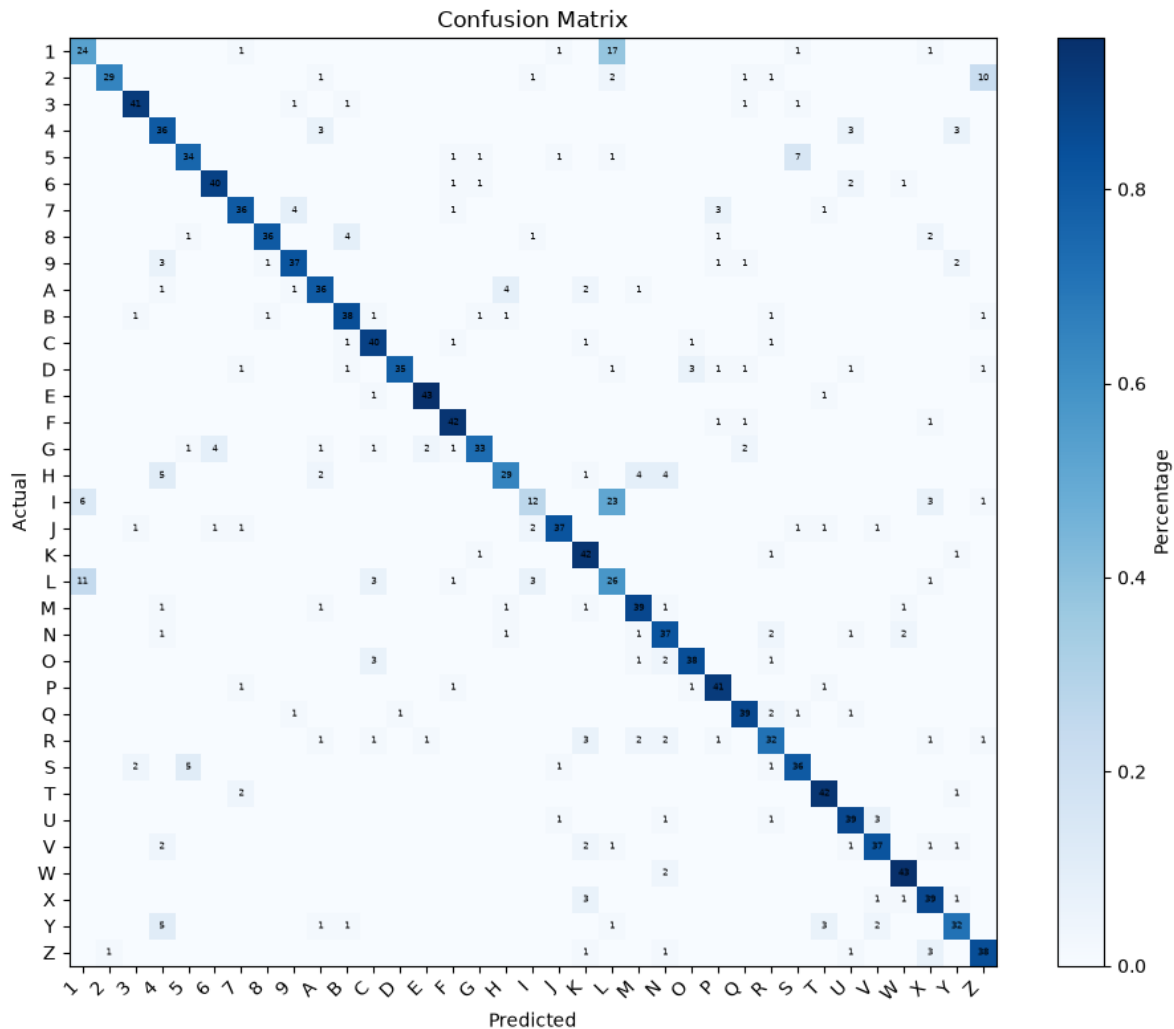
The held-out test set contains 1,575 examples, with 45 examples per class. It was used only after the final configuration was selected using validation performance.

Reported test accuracy: 79.87%. The validation accuracy of 80.4% and test accuracy of 79.87% are close, indicating consistent performance on unseen data and no major validation/test divergence.

Macro F1: 0.80. Because every test class has identical support (45 samples), the weighted F1 is mathematically equivalent to the macro F1 in this balanced test set.

13. Confusion Matrix

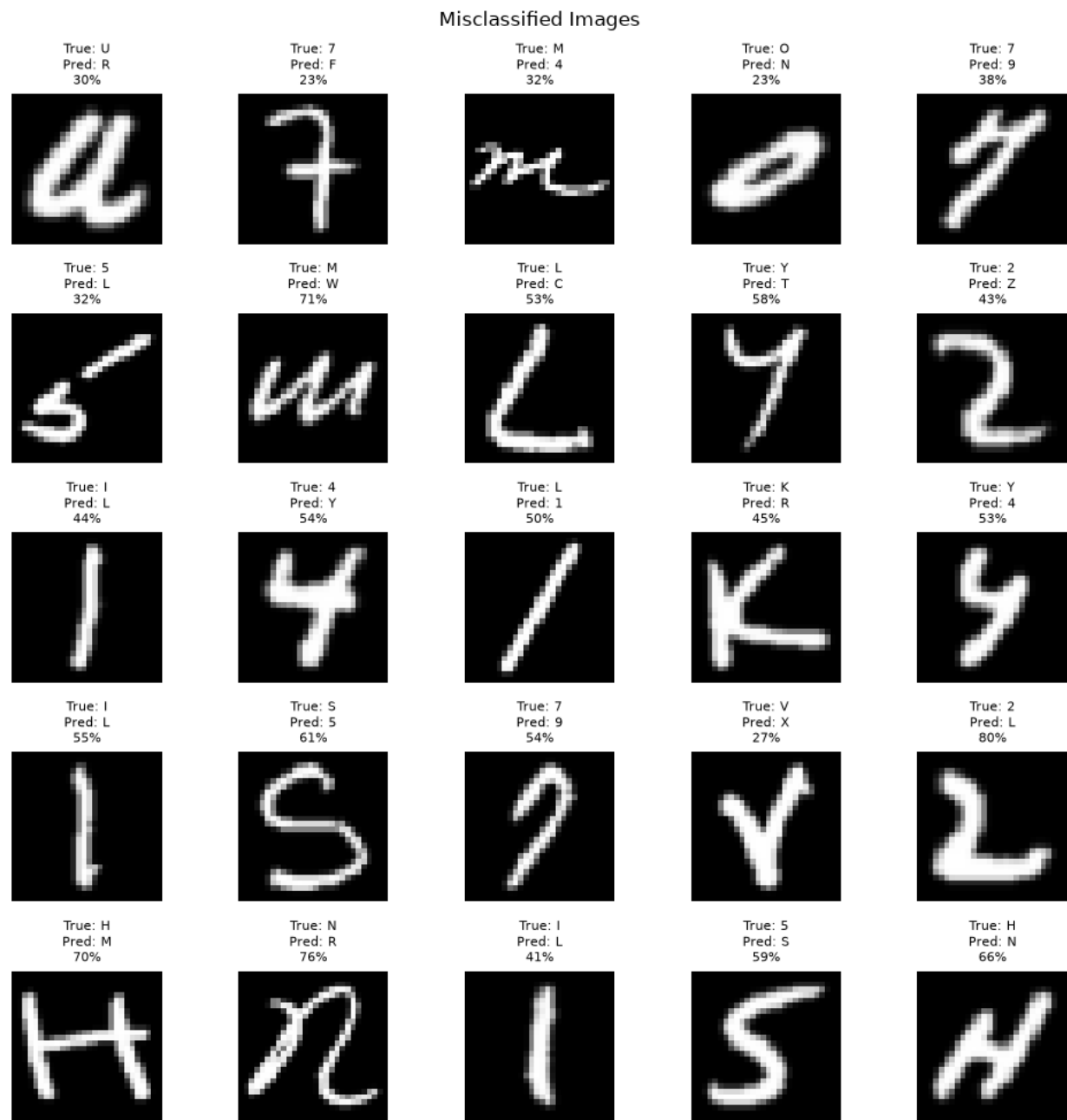
The confusion matrix shows the true class on the vertical axis and the predicted class on the horizontal axis. Strong values along the diagonal represent correct predictions. Off-diagonal concentrations show which characters are systematically confused.



14. Misclassified Examples and Error Analysis

The evaluation pipeline identifies incorrect predictions and visualizes the corresponding 28×28 images. Five representative errors were analyzed to connect

the numerical results with actual character shapes.



#	True	Predicted	Confidence	Interpretation
1	I	L	55%	Systematic I→L confusion; a simple vertical stroke dominates both characters, while secondary cues are weak.

2	2	L	80%	Confidently wrong; suggests a calibration issue on a visually ambiguous sample.
3	L	1	50%	Confirms bidirectional confusion within the I/L/1 ambiguity cluster.
4	7	F	23%	Low-confidence mistake consistent with visual similarity and uncertainty.
5	H	N	66%	Another structural-similarity error: two vertical strokes connected by a central structure.

The most important error pattern was the I/L/1 cluster. The repeated appearance of these confusions indicates a systematic feature-learning limitation rather than isolated random mistakes. A second recurring pattern involved structurally similar characters such as H and N.

15. Challenges Encountered and How They Were Resolved

Challenge	Impact	Resolution
Data orientation mismatch	Loader/preprocessing/split code initially used inconsistent row/column conventions.	Standardized the final pipeline to X=(784,m) and Y=(35,m), with samples as columns.
Label indexing confusion	One-hot encoding can be off by one when raw labels are 1-indexed.	Verified the raw labels explicitly and used labels-1 for NumPy zero-based indexing.
Randomness and	Shuffling and initialization	Introduced explicit seeds

reproducibility	could produce different results across runs.	and reused the same random generator within relevant stages.
Numerical stability in softmax	Large exponentials can overflow.	Subtracted the per-example maximum before exponentiating.
Gradient correctness	A model can appear to train even when gradients are subtly wrong.	Used shape checks and finite-difference numerical gradient checking.
Overfitting	Training accuracy greatly exceeded validation accuracy.	Measured the gap and compared multiple L2 values before selecting $\lambda=0.5$.
EMNIST image orientation	Raw images were not visually aligned with expected character orientation.	Verified representative samples manually and applied a transpose only for human-facing visualization.
Python module imports	Model and utility files lived in subfolders and were not always discovered by direct imports.	Added explicit local module paths so train/evaluate scripts can import the project modules reliably.

16. File-by-File Implementation Summary

Each file has a specific responsibility. Keeping the responsibilities separate made it easier to test individual components and locate errors.

File / Module	Purpose	Key finding / contribution
loader.py	Reads the raw CSV,	Confirmed the raw label

	separates labels from pixel values, and returns NumPy arrays.	convention and preserved the original pixel values for preprocessing.
preprocess.py	Handles grayscale checking, fixed-size handling, normalization, flattening, and one-hot labels.	Produces the final feature representation of 784 inputs and 35-class one-hot targets.
split.py	Creates stratified train/validation/test splits.	Verified exact 210/45/45 samples per class.
build_dataset.py	Runs loading, preprocessing, splitting, and saves processed arrays.	Creates a reusable, reproducible dataset artifact for downstream experiments.
layers.py	Initializes parameters, performs forward propagation, and performs generic L-layer backpropagation.	The network logic is architecture-independent; changing layer_dims changes depth/width without rewriting the algorithm.
activations.py	Implements ReLU, ReLU derivative, and softmax.	Stable softmax prevents exponential overflow; ReLU provides non-linearity.
loss.py	Computes cross-entropy, L2 penalty, and L2 gradient adjustment.	Separates regularization from the already-verified base backward pass.
optimizers.py	Implements GD, Momentum, RMSprop,	Allowed optimizer comparison; Adam

	and Adam.	became the final training optimizer.
batching.py	Shuffles examples and builds mini-batches.	Keeps X and Y aligned while allowing repeated parameter updates per epoch.
train.py	Main training orchestration.	Connects batching, forward pass, loss, backpropagation, regularization, Adam, and validation.
metrics.py	Computes confusion matrix, per-class precision/recall/F1, macro and weighted averages.	Turns raw predictions into class-level performance measures.
visualization.py	Creates the confusion-matrix heatmap and misclassified-image grid.	Makes systematic failure patterns visible to a reviewer.
evaluate.py	Loads the trained model and test set, runs the final evaluation, and saves reports/figures.	Produces final accuracy, per-class metrics, confusion matrix, and misclassified examples.

17. Key Findings

- Class balance was controlled rather than assumed: all 35 classes contain exactly 300 examples before splitting and 45 test examples after splitting.
- The L-layer design was more valuable than hardcoding the chosen architecture because it made the implementation reusable and reduced duplicate forward/backward code.

- The baseline model clearly overfit. The train/validation gap provided a measurable reason to introduce regularization instead of adding it without evidence.
- L2 regularization at $\lambda=0.5$ provided the best balance among the tested values: the train/validation gap dropped substantially without a large loss in validation accuracy.
- Numerical gradient checking gave relative error around $1e-8$, providing direct evidence that the analytical backpropagation was implemented correctly.
- The close validation and test accuracies (80.4% and 79.87%) indicate that the final model generalizes consistently to unseen data.
- The main remaining weakness is systematic confusion among visually similar, stroke-simple characters, especially I, L, and 1.
- The project exposed a real image-orientation issue during visualization, demonstrating the value of inspecting actual samples instead of trusting numerical pipelines blindly.

18. Limitations

- The numerical gradient check sampled 50 elements of the largest $W1$ matrix instead of checking all 12,544 values; smaller parameter arrays were checked exhaustively.
- The regularization sweep tested four λ values (0, 0.1, 0.5, 1.0) rather than a dense search.
- Only one train/validation/test split was used rather than k-fold cross-validation.
- The final model is a fully-connected network. Convolutional structure could better exploit the spatial relationships inside 28×28 images, but CNNs were outside the assignment constraint and would require a different implementation scope.
- The dataset intentionally uses a relatively small, balanced sample count of 300 per class. Performance on a much larger and more naturally distributed handwriting dataset may differ.

19. Future Improvements

- Experiment with additional hidden-layer widths and depths using the existing L-layer infrastructure.

- Introduce controlled image augmentation such as small translations or rotations while keeping labels unchanged.
- Explore better calibration of softmax confidence, especially for high-confidence wrong predictions.
- Investigate the I/L/1 confusion cluster with targeted class-specific preprocessing or additional training examples.
- Add a systematic experiment-tracking file containing seed, optimizer, learning rate, regularization value, epoch count, and final metrics for each run.

20. Reproducibility and Project Organization

The project uses project-relative paths rather than machine-specific absolute paths. Processed data and trained parameters are stored as NumPy .npy files, while evaluation outputs are written into a report/figures directory. This structure allows the same codebase to be moved or cloned without editing hardcoded local paths.

The implementation is modular: data loading, preprocessing, splitting, model mathematics, optimization, batching, evaluation metrics, visualization, training, and final evaluation are separated into individual files. This separation also made debugging more manageable because failures could be traced to a specific stage.

21. Conclusion

The project successfully implemented a complete character-recognition neural network from scratch using NumPy only. The work went beyond the minimum classification requirement by adding a reusable L-layer design, multiple optimization algorithms, numerical gradient verification, L2 regularization experiments, balanced stratified splitting, detailed class-level metrics, and visual error analysis.

The most important engineering lesson from the project was that a model is not validated simply because loss decreases. Data integrity, shape conventions, reproducibility, gradient correctness, overfitting behavior, and qualitative inspection of errors all required explicit verification. The resulting system is therefore not only a working classifier, but also a transparent implementation in which each major stage can be inspected, tested, and improved independently.

Appendix A. Core Equations Used

Component	Equation
Dense layer	$Z[l] = W[l] \cdot A[l-1] + b[l]$
ReLU	$\text{ReLU}(x) = \max(0, x)$
ReLU derivative	$\text{ReLU}'(x) = 0 \text{ for } x \leq 0; 1 \text{ for } x > 0$
Stable softmax	$\text{softmax}(z) = \exp(z - \max(z)) / \sum \exp(z - \max(z))$
Cross-entropy	$\text{Loss} = -(1/m) \sum Y \cdot \log(Y_{\text{hat}})$
Output gradient	$dZ[L] = A[L] - Y$
Weight gradient	$dW[l] = (1/m) dZ[l] \cdot A[l-1]^T$
Bias gradient	$db[l] = (1/m) \sum dZ[l]$
L2 penalty	$L2 = (\text{lambda}/(2m)) \sum W ^2$
L2 gradient adjustment	$dW \leftarrow dW + (\text{lambda}/m)W$

End of Report